

Song of the Day (Spotify): A Lightweight, Novelty-Aware and Mood-Conditioned Music Recommender

Silas Spencer

August 11, 2025

Abstract

We present *Song of the Day* (SotD), a lightweight music recommender that surfaces exactly one *Song of the Day* and five alternates per day using a user’s Spotify history. The system blends four objectives: (1) novelty—to reduce repetition and emphasize discovery; (2) user fit—to respect a user’s long-term taste; (3) optional mood alignment—via a valence–arousal mapping onto Spotify audio features; and (4) diversity—to avoid redundant alternates. SotD is implemented in Python (Poetry-managed), leverages the Spotify Web API via `spotipy`, uses `pandas/numpy/scikit-learn` for data processing and modeling, provides a quick UI in Streamlit, and persists lightweight caches in SQLite. We describe the design, scoring model, re-ranking scheme, implementation details, and challenges (API limits, bias, mood interpretation, and privacy). We include a reproducible CLI and outline evaluation criteria and future work (context-aware factors, reinforcement learning re-ranking, and richer explanations).

1 Introduction

Daily recommendation experiences (e.g., “song of the day”) provide a focused ritual that can improve perceived quality and reduce choice overload. While modern streaming platforms excel at large-scale recommendation, power users often desire controllable, inspectable, and reproducible tooling on their own data. SotD aims to be:

- **Minimal:** one canonical pick + five alternates.
- **Balanced:** novelty, fit, optional mood, and diversity.
- **Local and transparent:** small stack, human-readable rationale.

2 Background and Foundation

2.1 Recommenders and Multi-Objective Trade-offs

Recommender systems frequently optimize a blend of accuracy and beyond-accuracy goals such as novelty, serendipity, and diversity. In music, over-emphasizing accuracy can lead to *feedback loops* and recommendation collapse; over-emphasizing novelty can decrease satisfaction. SotD targets the *daily* granularity, enabling a meaningful blend at a small slate size ($1 + 5$).

2.2 Valence–Arousal Mood Space

To optionally align recommendations with self-reported mood, we use the circumplex model (valence: positive/negative; arousal: energetic/calm). Spotify provides **valence** and proxies for arousal (e.g., **energy**, **tempo**); we map mood prompts (e.g., “chill”) to a target (v^*, a^*) and measure distance to track features.

2.3 Data Sources

We use the Spotify Web API via `spotipy` to retrieve:

- Recent plays, top artists, and top tracks/genres (short/medium/long term).
- Track-level audio features (valence, energy, tempo, danceability, etc.).

A local SQLite cache minimizes repeated network calls and speeds up daily runs.

3 System Overview

3.1 Objectives

Given candidate tracks $\mathcal{C}$ derived from a user’s listening graph, we compute a composite score

$$S(t) = w_n \underbrace{\text{Nov}(t)}_{\text{novelty}} + w_f \underbrace{\text{Fit}(t)}_{\text{taste fit}} + w_m \underbrace{\text{Mood}(t)}_{\text{valence-arousal}} + w_d \underbrace{\text{Div}(t|\mathcal{S})}_{\text{diversity in re-rank}}, \quad (1)$$

where $\mathcal{S}$ is the selected set (empty for the primary pick, non-empty during alternates re-ranking). We output the top-1 as the *Song of the Day* and greedily re-rank for five diverse alternates.

3.2 Data Flow

1. **Ingestion:** recent tracks, top artists/tracks, and audio features.
2. **Caching:** persisted in SQLite at `./data/sotd_cache.db`.
3. **Featureization:** compute novelty features (recency, lifetime usage), user taste vectors, mood target (v^*, a^*) .
4. **Candidate generation:** merge recent items, items from top artists/genres, and optionally unseen catalog from followed artists.
5. **Scoring and re-ranking:** compute $S(t)$ and perform diversity-aware greedy selection.
6. **Presentation:** CLI or Streamlit UI shows the pick, alternates, and rationale.

4 Methodology

4.1 Novelty

We penalize over-exposed items using both short-term recency and lifetime plays:

$$\text{Nov}(t) = \alpha \cdot \exp\left(-\frac{\Delta\text{days}(t)}{\tau}\right) + (1 - \alpha) \cdot \left(1 - \frac{\text{Plays}_{\text{lifetime}}(t)}{\max_p + \epsilon}\right), \quad (2)$$

where $\Delta\text{days}(t)$ is days since last play, τ is a decay constant, and $\max_p$ normalizes lifetime plays. This rewards “not recently played” and “less saturated” tracks.

4.2 User Fit

We embed users and tracks in a feature space combining audio features and (optionally) artist/genre one-hot or embeddings. Let $u \in \mathbb{R}^d$ denote a user taste vector (e.g., average of historical track vectors with recency weighting) and $x_t \in \mathbb{R}^d$ the track vector. Fit is cosine similarity:

$$\text{Fit}(t) = \frac{u^\top x_t}{\|u\| \|x_t\|}. \quad (3)$$

4.3 Mood Alignment (Optional)

Map a mood prompt to a target (v^*, a^*) ; we construct a track arousal proxy a_t using a normalized combination of **energy**, **tempo**, and **danceability**, and use Spotify **valence** as v_t . Mood match is the negative distance:

$$\text{Mood}(t) = 1 - \sqrt{(v_t - v^*)^2 + (a_t - a^*)^2}. \quad (4)$$

If no mood is provided, set $w_m = 0$.

4.4 Diversity via Greedy Re-Ranking

We use Maximal Marginal Relevance (MMR)-style re-ranking for alternates:

$$\text{MMR}(t | \mathcal{S}) = \lambda \tilde{S}(t) - (1 - \lambda) \max_{s \in \mathcal{S}} \text{sim}(x_t, x_s), \quad (5)$$

where $\tilde{S}$ is min-max normalized base score from Eq. (1) with $w_d=0$, and sim is cosine similarity. We iteratively add the item with highest MMR until five alternates are chosen.

4.5 End-to-End Algorithm (Greedy)

Inputs: user history $\mathcal{H}$, audio features, optional mood (v^*, a^*) , weights (w_n, w_f, w_m) , λ , alternates $k=5$.

1. Build candidates $\mathcal{C}$ from recent tracks, top artists/genres (plus unseen items).
2. Compute $\text{Nov}(t)$, $\text{Fit}(t)$, and $\text{Mood}(t)$ for $t \in \mathcal{C}$.
3. Compute base score $S_0(t) = w_n \text{Nov}(t) + w_f \text{Fit}(t) + w_m \text{Mood}(t)$.
4. Pick $t^* = \arg \max_{t \in \mathcal{C}} S_0(t)$ as the Song of the Day. Set $\mathcal{S} \leftarrow \{t^*\}$.
5. For $i = 1$ to k : select $t_i = \arg \max_{t \in \mathcal{C} \setminus \mathcal{S}} \text{MMR}(t | \mathcal{S})$ and update $\mathcal{S} \leftarrow \mathcal{S} \cup \{t_i\}$.
6. Return t^* and $\{t_1, \dots, t_k\}$ with per-component contributions as rationale.

5 Implementation

5.1 Stack

- **Language/Tooling:** Python 3.11, Poetry.
- **Libraries:** spotipy, pandas, numpy, scikit-learn.
- **UI:** Streamlit (quick daily UI).
- **Storage:** SQLite (`./data/sotd_cache.db`) for token, profile, and feature caches.

5.2 Reproducibility & Setup

Install Poetry (macOS):

```
brew install poetry
```

Or:

```
curl -sSL https://install.python-poetry.org | python3 -
```

Environment:

```
cp example.env .env
# Fill SPOTIFY_CLIENT_ID, SPOTIFY_CLIENT_SECRET, SPOTIFY_REDIRECT_URI, SPOTIFY_USERNAME
```

Dependencies:

```
poetry install
# Optional shell
poetry shell
```

CLI:

```
poetry run song-of-the-day --mood "chill" --limit 20
```

Streamlit:

```
poetry run streamlit run app/streamlit_app.py
```

5.3 Caching and Rate Limits

All API responses (user profile slices, audio features) are cached in SQLite with timestamps. We invalidate stale segments (e.g., recent plays) based on configurable TTLs and batch audio-feature queries to respect Spotify rate limits.

6 Evaluation

SotD is built for personal usage; nevertheless, we outline evaluation axes suitable for small-scale studies:

- **Repetition control:** proportion of days where the daily pick is not in the last- N days of history.
- **Diversity:** intra-list diversity among the five alternates (average pairwise dissimilarity).
- **Mood fidelity:** correlation between target (v^*, a^*) and selected items' (v_t, a_t) .
- **User satisfaction:** Likert feedback or thumbs-up on daily picks.
- **Coverage/novel discovery:** fraction of picks that were previously unplayed or from underrepresented artists/genres.

A pragmatic approach is an A/B over several weeks varying (w_n, w_f, w_m, λ) and analyzing engagement and feedback.

7 Challenges and Limitations

API Constraints. Rate limits and pagination require batching and caching; new users with sparse data can face cold-start issues.

Bias and History Effects. Top-artist seeding can entrench past preferences; we mitigate via novelty penalties and diversity-aware alternates.

Mood Interpretation. Mapping free-text mood to (v^*, a^*) is lossy; synonyms and cultural context can misalign with audio features.

Explainability. Presenting clear rationale (e.g., novelty gain vs. fit trade-off) is essential to trust; we include per-component score breakdowns.

8 Ethical and Privacy Considerations

User listening data are personal. SotD only accesses scopes strictly required for recommendation, stores minimal derived features locally, and supports revoking tokens. Rationale should avoid revealing sensitive insights (e.g., inferred demographics). We recommend transparency about what is cached and why.

9 Future Work

- **Context-aware features:** time-of-day, weekday, weather, and activity cues to improve fit.
- **Learning to re-rank:** bandits or RL (e.g., MAB/MMR hybrids) using implicit feedback on daily picks.
- **Richer embeddings:** CLAP/semantic-audio or graph-based artist embeddings for better similarity.
- **Better mood interface:** few-shot mapping from free text to (v^*, a^*) with calibration.
- **Explanations:** natural-language rationales summarizing novelty, mood, and fit in a concise blurb.

10 Conclusion

We introduced a pragmatic, local-first recommender that balances novelty, fit, optional mood, and diversity to produce a single “Song of the Day” with five alternates. With a compact stack and transparent scoring, SotD offers an inspectable daily ritual that encourages discovery without abandoning personal taste. We outlined design choices, scoring, re-ranking, and evaluation criteria, and discussed limitations and future enhancements.

Availability. The system uses Poetry for reproducibility, a CLI for automation, and a Streamlit app for quick interaction. The cache (`./data/sotd_cache.db`) is created on first run. Ensure the Spotify Redirect URI matches `.env` (default `http://localhost:8080/callback`).

References

- [1] Spotify Web API. *Spotify for Developers*. <https://developer.spotify.com/documentation/web-api>
- [2] J. A. Russell. A circumplex model of affect. *Journal of Personality and Social Psychology*, 39(6):1161–1178, 1980.
- [3] J. Bobadilla, F. Ortega, A. Hernando, and A. Gutiérrez. Recommender systems survey. *Knowledge-Based Systems*, 46:109–132, 2013.

- [4] F. Pedregosa et al. Scikit-learn: Machine learning in Python. *JMLR*, 12:2825–2830, 2011.
- [5] Streamlit, Inc. Streamlit Documentation. <https://docs.streamlit.io>